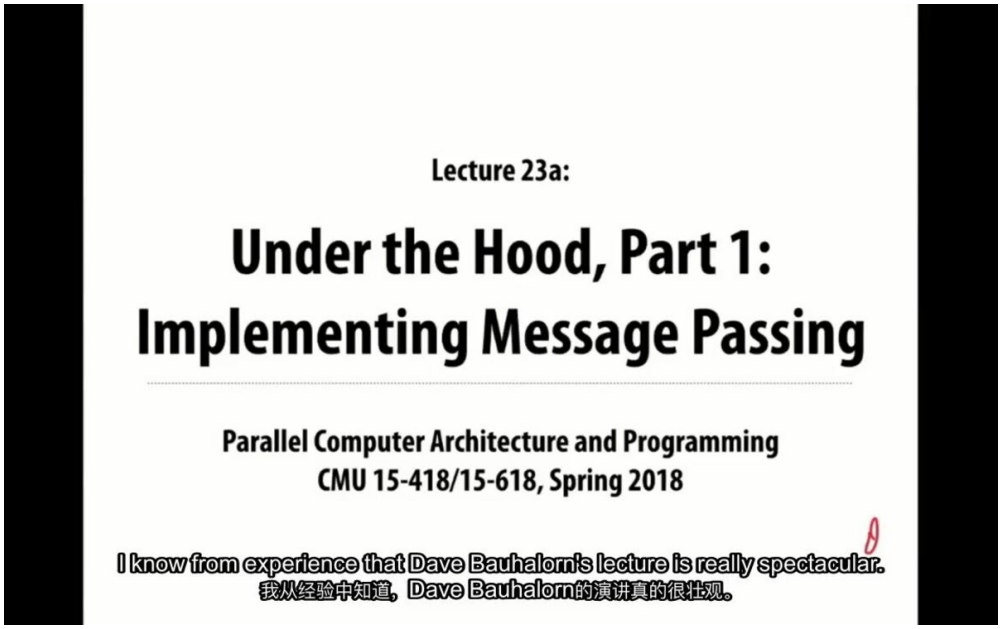
课程笔记

消息传递的实现：缓冲、流控与信用机制

CMU 15-418/618 并行计算机架构与编程 · Lecture 27（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 19 分 10 秒
视频链接：`https://www.bilibili.com/video/BV1Gh4y1S75H/`

目录

1 消息传递为什么容易实现 **2**
1.1 基本操作 2
1.2 硬件上为什么受欢迎 3
1.3 本章小结 4

2 与共享地址空间的对比 **4**
2.1 共享地址空间怎么搬数据 4
2.2 关键差别 5
2.3 本章小结 5

3 两种语义：同步与异步 **6**
3.1 直觉的陷阱 6
3.2 消息的尺寸 7
3.3 本章小结 7

4 eager 协议：三段流程 **8**
4.1 先发通知，不发数据 8
4.2 本章小结 8

5 同步协议的三段流程 **9**
5.1 回应地址 9
5.2 像 DMA 一样传输 9
5.3 本章小结 10

6 异步的麻烦与流控 **10**
6.1 发送方阻塞反而是好事 10
6.2 接收方的问题难得多 11
6.3 本章小结 12

7 信用式混合方案 **12**
7.1 预分配与通告 12
7.2 跟踪用量 12
7.3 本章小结 14

8 总结与延伸 **14**

1　消息传递为什么容易实现

这一讲讲所用框架的内部实现。讲者先用约 25 分钟讲消息传递实现中数据的实际搬运，然后交给另一位教授讲其他内容。

1.1　基本操作

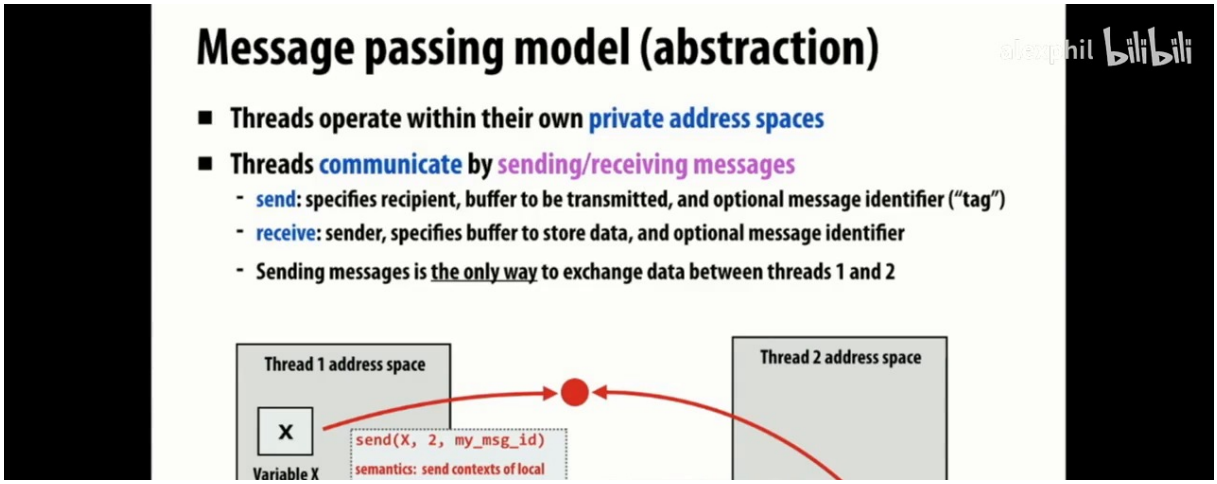


图 1：消息传递模型：线程在各自的私有地址空间中操作[1]

讲者回顾：大家现在既有共享地址空间的编程经验，也有消息传递的经验。消息传递中的原语通信操作，就是把一条消息从一个处理器发到另一个处理器。

消息传递的复制开销无法回避

共享地址空间可以靠缓存让多个处理器共享同一份数据；消息传递每次通信都要复制。因此消息传递的成本与通信量成正比，而共享地址空间的成本与缓存行迁移次数成正比。这决定了它们在不同计算通信比下的优劣。

发送方执行 send，接收方执行 receive。这个操作最终把数据从发送方的私有地址空间复制到接收方私有地址空间的某一部分。

[1] 视频画面时间区间：00:01:00–00:01:15。若为裁剪图，时间区间仍对应原视频画面。

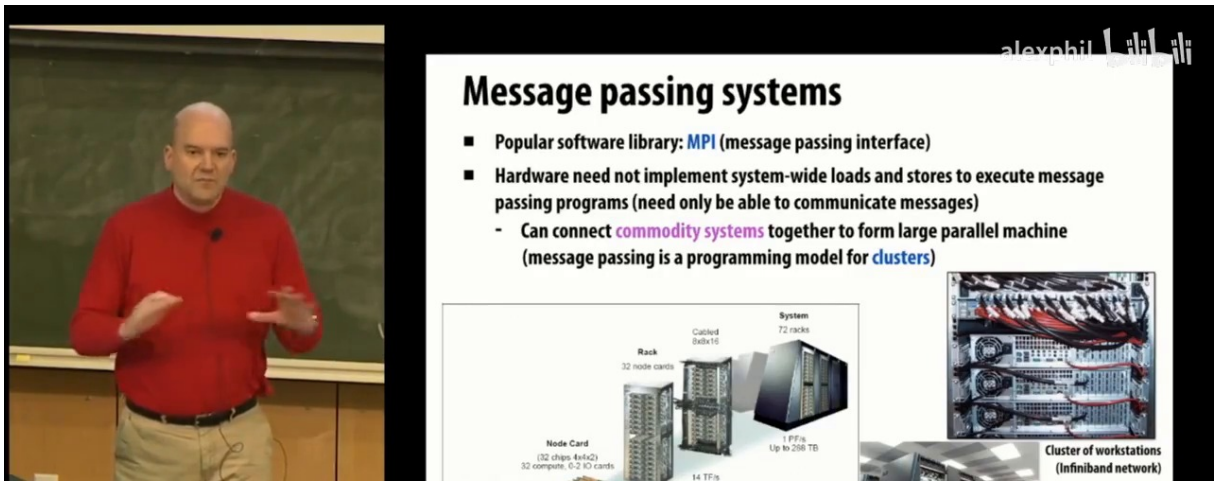


图 2: 消息传递系统：流行库为 MPI，硬件需求很少[2]

1.2 硬件上为什么受欢迎

讲者给出一个很实际的理由：市面上很多系统采用消息传递，原因很简单——搭建大规模消息传递计算机非常容易，因为你除了一个足够快的互连之外不需要任何特殊硬件。

特别是与共享地址空间相比，你不需要缓存一致性，也不需要内存系统的任何特殊支持。

消息传递的可扩展性上界由互连决定

既然不需要一致性硬件，那么规模上限就只取决于互连的带宽与延迟。这也意味着消息传递系统的性能瓶颈几乎总是网络，而不是处理器。因此第 19 讲的拓扑与对分带宽分析在这里直接适用。

他甚至说：理论上你可以把手机、物联网设备之类全都拉进来，只要它们能通过某个（希望足够快的）网络互相发消息，就构成一台大的并行计算机。

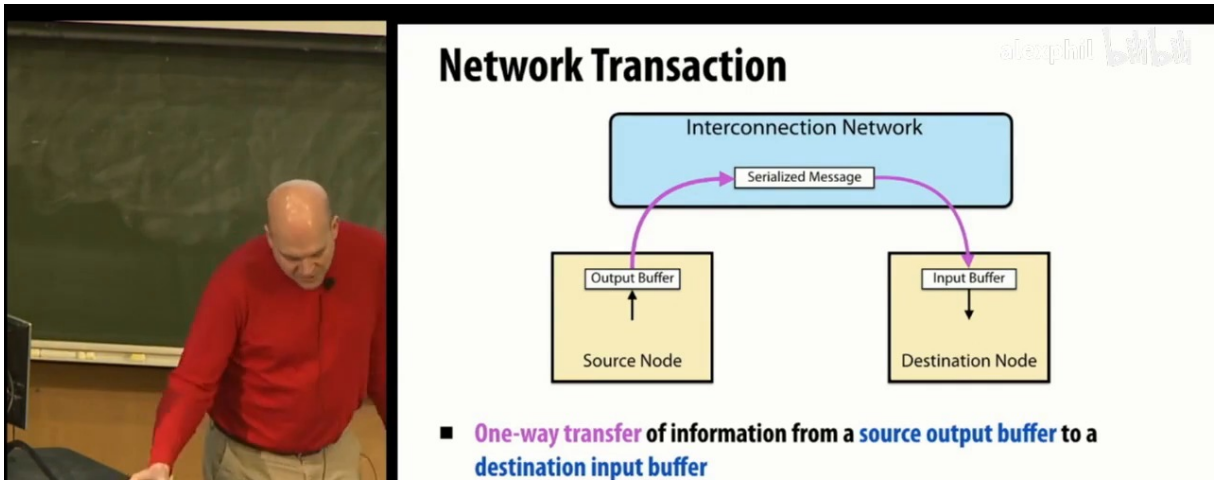


图 3: 网络事务：序列化消息、输出缓冲、输入缓冲与互连网络[3]

[2] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

[3] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

消息传递的规模优势来自”不需要一致性”

共享地址空间要跨节点就必须维护缓存一致性，而一致性协议的复杂度与节点数密切相关（见第 11、13 讲）。消息传递把这层复杂度完全移出硬件，因此可以靠增加节点线性扩展。代价是软件必须显式管理数据放置。

在整个并行计算历史上，这都是搭建大型机器相当流行的方式。今天在云计算中，很多服务器就是这样工作的：独立的刀片或节点之间并没有缓存一致性，因此让软件跑起来最容易的方式就是消息传递。

讲者补了一句很关键的限定：它受欢迎并不一定因为它是程序员写软件时最方便的首选方式——你们做完作业可能各有看法——但从硬件角度看它确实很棒。

”硬件容易”与”编程容易”是两条独立的评价轴

消息传递不需要缓存一致性，因此搭建大规模机器几乎只依赖互连；但程序员必须显式搬运数据、显式安排同步。因此它是一套”硬件成本低、软件成本高”的方案。这个不对称正是它在大规模系统中长期占优的原因。

1.3 本章小结

消息传递的原语是把数据从发送方私有地址空间复制到接收方。它在硬件上极受欢迎：只需足够快的互连，不需要缓存一致性或特殊支持，因此云计算的刀片间也用它。但它未必是程序员最方便的写法——这是”硬件成本低、软件成本高”的不对称。

2 与共享地址空间的对比

2.1 共享地址空间怎么搬数据

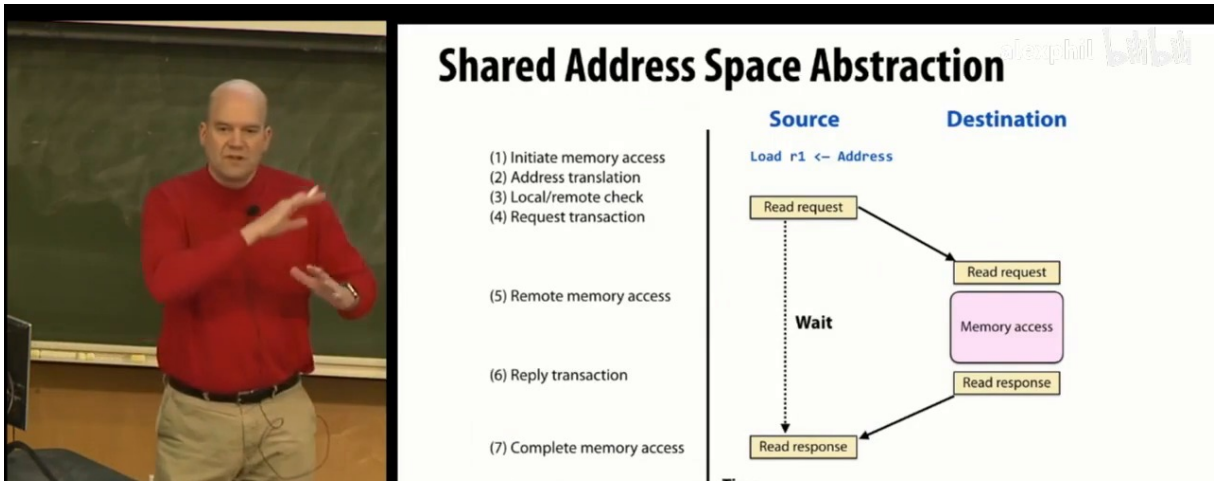


图 4: 共享地址空间抽象：发起访问、地址转换、取回数据[4]

[4] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

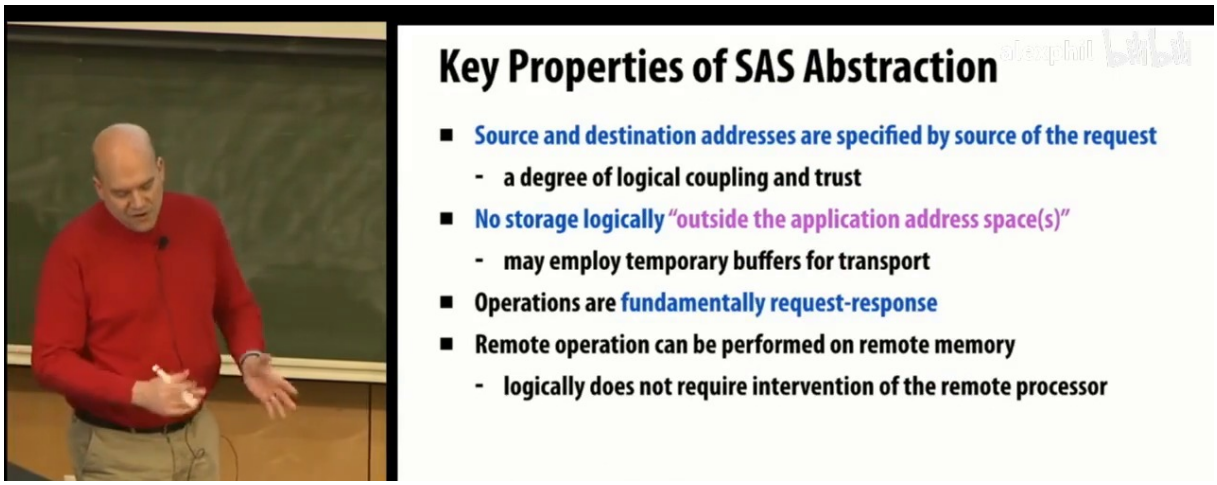


图 5: 共享地址空间的关键性质：地址由请求方指定[5]

讲者先展示共享地址空间下数据的实际搬运，以便与消息传递对比。

有趣之处在于：在共享地址空间中，数据主要通过 ** 读 ** 被搬进消费方的缓存。写操作只是更新数据，真正把数据拉过来的是读。

共享地址空间把”数据在哪”这个问题交给硬件

请求方只给出地址，其余由地址转换与一致性协议完成。因此程序员不必知道数据实际位于哪个节点的内存。消息传递没有这层机制，因此”数据在哪”必须由程序自己跟踪。

一次读会把某个虚拟地址转成物理地址，硬件据此知道去哪里取数据。

2.2 关键差别

讲者指出一个当时看起来不明显、但后面会冒出来的要点：共享地址空间不需要在地址空间之外的额外缓冲或存储。

它本质上是一个请求-响应协议，这让它简单。

共享地址空间的隐含假设是”地址双方都知道”

在共享地址空间中，请求方只需给出地址，硬件负责把它翻译成物理位置并取回数据。发送方不需要知道接收方的缓冲区在哪，接收方也不需要预先安排。消息传递没有这个前提，因此必须显式交换地址或复制数据。

2.3 本章小结

共享地址空间靠读把数据拉进消费方缓存，写只更新数据；读时由硬件做地址转换。它不需要地址空间之外的额外缓冲，本质是请求-响应协议，因此简单。其隐含前提是双方都能通过地址访问同一份数据，而消息传递没有这个前提。

[5]视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

3　两种语义：同步与异步

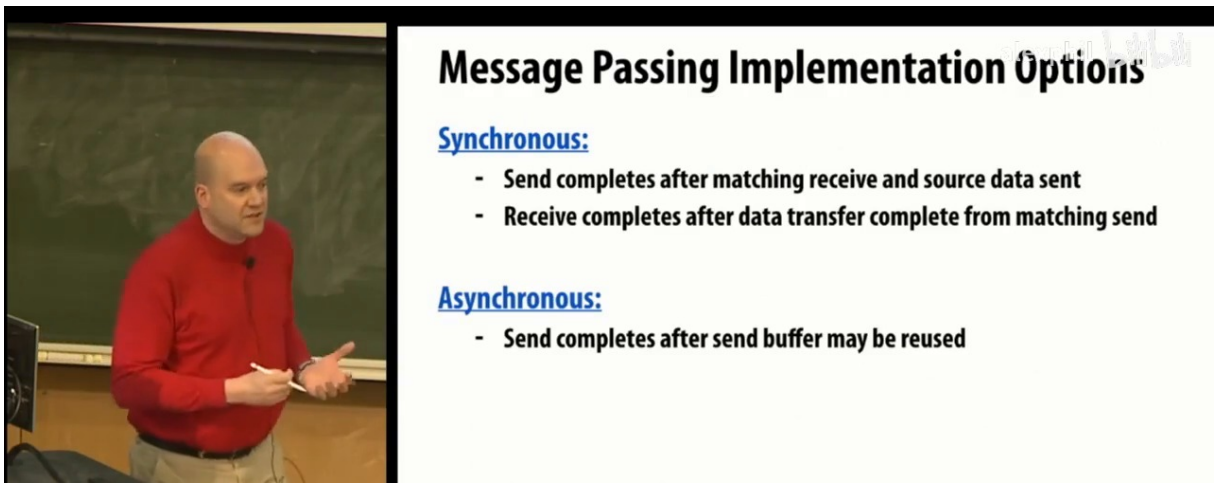


图 6: 消息传递的实现选项：同步与异步[6]

讲者回顾几周前看网格求解代码时引入的概念。

同步消息传递：发送方停顿，直到接收方已经收到。

异步消息传递：你发出去、把它缓冲起来、继续干自己的事，最终接收方会收到它，然后双方各自继续。

3.1　直觉的陷阱

讲者说，在完全不了解底层发生什么的情况下，如果只听说这两种，你的直觉大概会说：异步听起来好得多，因为它不让发送方停顿。发送方发消息时凭什么要停？我们宁愿继续跑。

异步的”不停顿”是有条件的

异步发送之所以不停顿，是因为消息被暂存在某处。若暂存空间耗尽，发送方仍会被迫阻塞。因此异步不是”永不阻塞”，而是”把阻塞推迟并集中到缓冲耗尽时”。理解这一点才能理解后面两种协议的取舍。

[6]视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

3.2 消息的尺寸

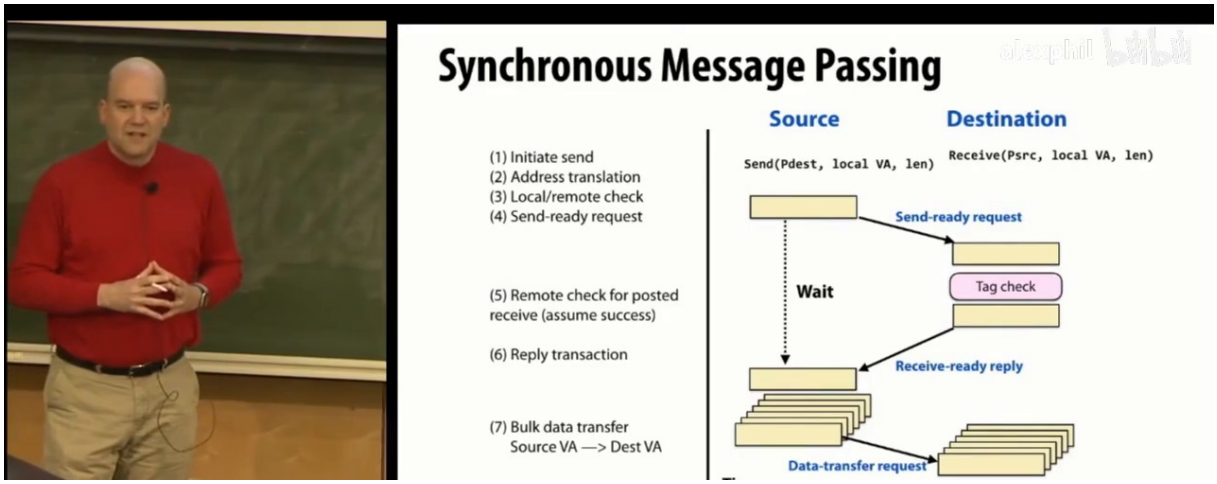


图 7: 同步消息传递：发起发送与匹配接收[7]

讲者提醒消息不只是单个缓存行：它们可能是 1024 字节这样的千字节量级，甚至兆字节级的数据。

而且课上早先讲过：发送大消息是好主意，你们大概也亲身经历过。发很小的消息——单个字、单个字节——会带来大量开销。更好的做法是把要发的数据合并成大消息。

因此系统必须支持传输任意长度的消息，而且它们可能很大。

3.3 本章小结

同步消息传递让发送方停顿到接收方收到；异步则缓冲后继续。直觉会偏向异步，但异步的”不停顿”以缓冲空间为前提。消息可能达千字节甚至兆字节，小消息开销大，因此应合并成大消息，而系统必须支持任意长度。

[7]视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

4 eager 协议：三段流程

4.1 先发通知，不发数据

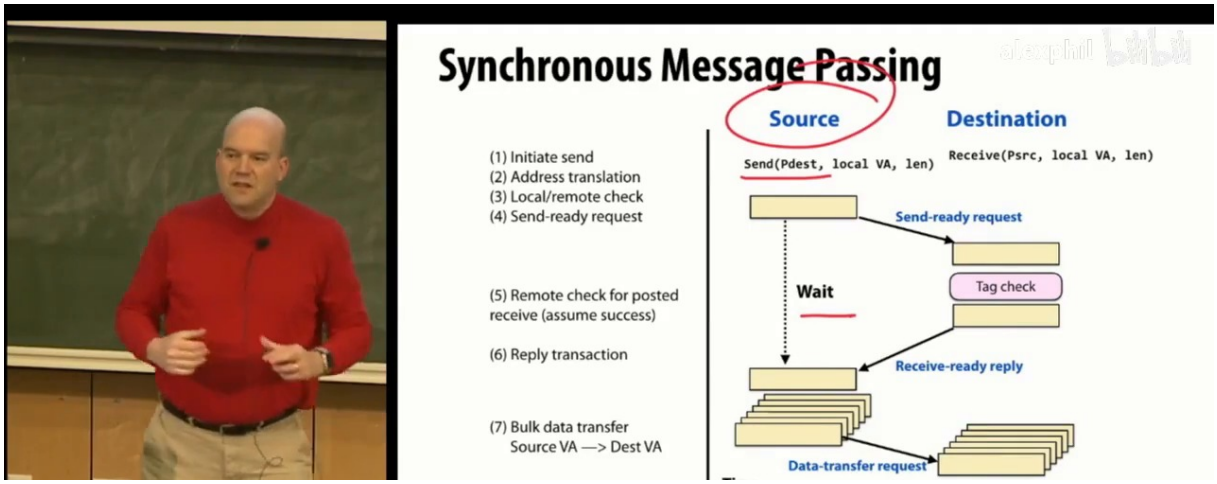


图 8: 同步消息传递续：发送方先通知而不发数据[8]

讲者用图示逐步说明。发送方想发消息，于是执行 send。

它先要做的是通知接收方” 我想发一条消息”，但此时还不发送消息里的实际数据。

这个通知要带上消息类型之类的信息，以便与接收方正等待的消息类型匹配。

4.2 本章小结

同步消息传递让发送方停顿到接收方收到；异步则缓冲后继续。直觉偏向异步，但异步的「不停顿」以缓冲空间为前提。消息可能达千字节甚至兆字节，小消息开销大，因此应合并成大消息，而系统必须支持任意长度。

下面看两种协议各自如何组织数据的实际搬运。

[8] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

5 同步协议的三段流程

5.1 回应地址

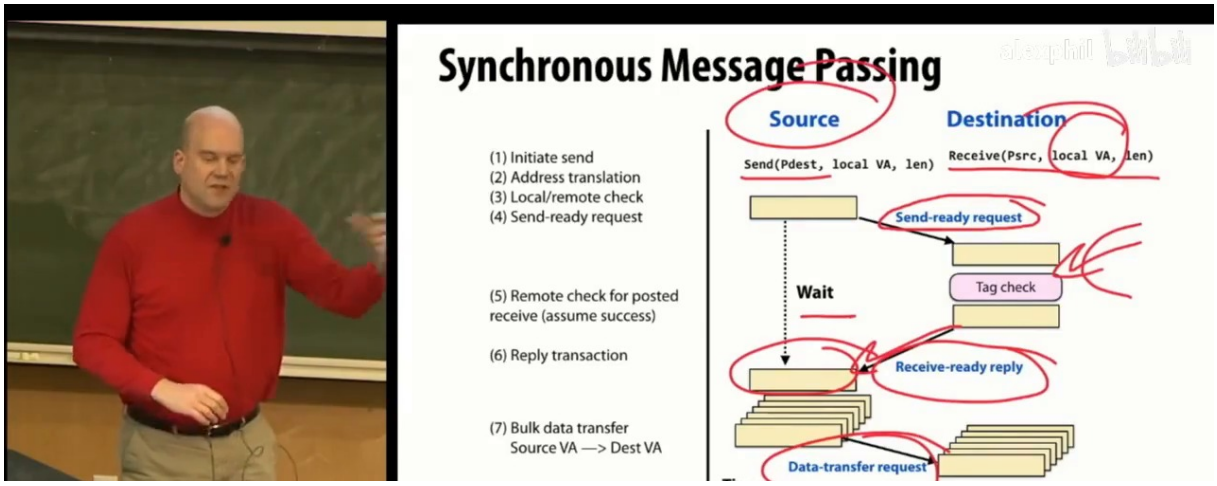


图 9: 同步消息传递续：接收方回应并告知目标地址[9]

假设接收方已经执行了一个匹配的 receive，那么它已经准备好、也确实想要这条消息。

于是它把一条应答消息发回给发送方，说” 我们准备好接收了”，并顺便告诉它：这是目标地址，数据传过来时应该放到这里。

5.2 像 DMA 一样传输

现在发送方知道了目的地址，可以开始传输数据。讲者说好处是：这时它就只是一次 DMA 传输——直接内存访问传输。

它只是把数据从发送方地址空间中一个已知位置，复制到接收方地址空间中一个已知位置，因为发送方已经知道目的地址。所以这就是内存之间的复制。

而这个方案的美妙之处在于：不需要在发送方与接收方地址空间之外的任何额外缓冲。

$$\text{跳数} = 1_{\text{通知}} + 1_{\text{回应地址}} + 1_{\text{数据}} = 3$$

- 第一跳：发送方把通知送到接收方；
- 第二跳：接收方把目标地址送回发送方；
- 第三跳：数据从发送方直接写入接收方地址空间；
- 因此即使是极小的消息，也至少需要三跳。

零额外缓冲换来的是往返延迟

这个协议不在地址空间外占用缓冲，代价是每次发送都要一次完整往返才能开始传数据。因此它的延迟下限由三跳决定。对延迟敏感的通信（如反复传递一个标志位），这个下限会成为瓶颈。

[9] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

5.3 本章小结

eager 协议分三段：发送方先发通知（含消息类型），接收方匹配后回应并告知目标地址，然后数据以 DMA 方式从已知位置复制到已知位置。它不需要地址空间外的额外缓冲，代价是至少三跳的往返延迟。

6 异步的麻烦与流控

6.1 发送方阻塞反而是好事

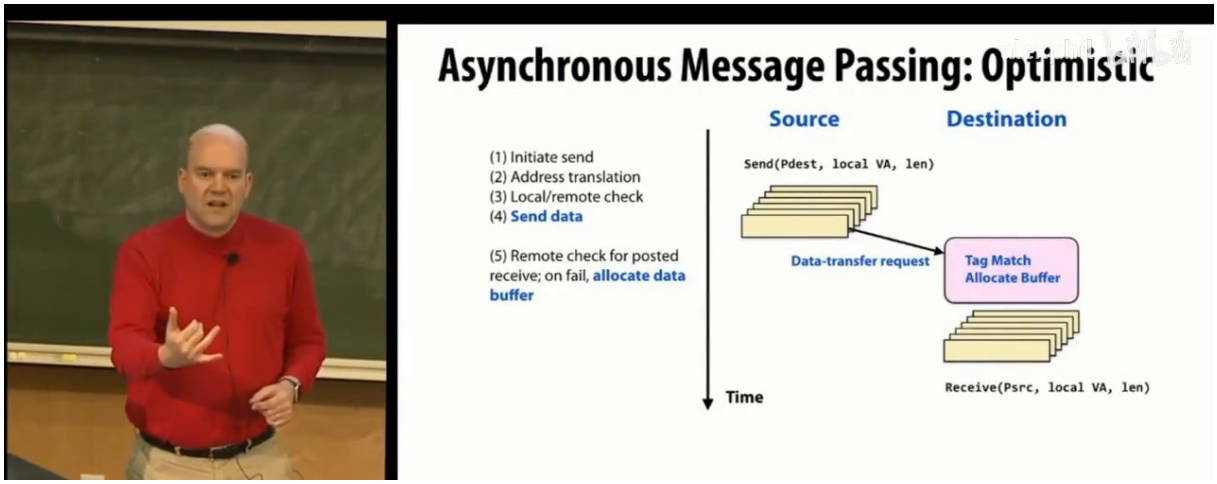


图 10: 异步消息传递（乐观）：发送后缓冲并继续[10]

讲者接着分析乐观的异步方案会遇到什么问题。

如果发送方缓冲用尽，它就停顿、阻塞，直到那些消息消失。这实际上是一种自然的流控——它把发送方节流回去，于是它们的发送不再真正异步，而会阻塞一段时间。

但

本地可判定性是分布式算法设计的关键

一个机制若能在本地判定是否需要等待，就不需要全局协调，因此实现简单、扩展性好。发送方缓冲检查满足这个条件；接收方的涌入问题不满足，因此必须靠预先约定的信用额度来转化。

讲者说这其实是容易处理的问题：因为你在发送方本地就能察觉”糟糕，缓冲用完了”，那好，我现在不再往消息层塞东西，就这么阻塞着，最终它会清空。

[10] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

6.2 接收方的问题难得多

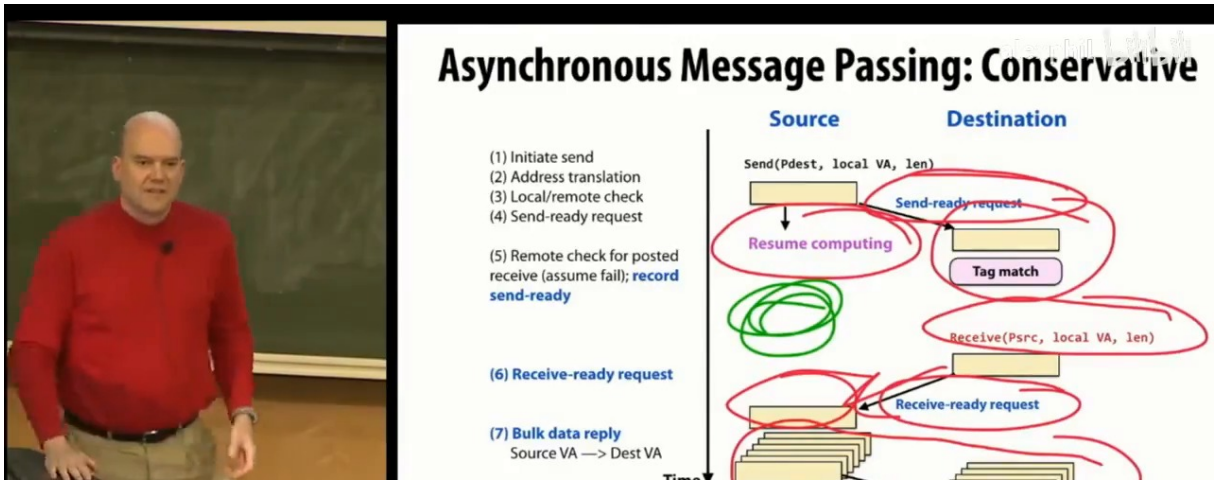


图 11: 异步（保守）方案：地址转换后确认接收方已就绪[11]

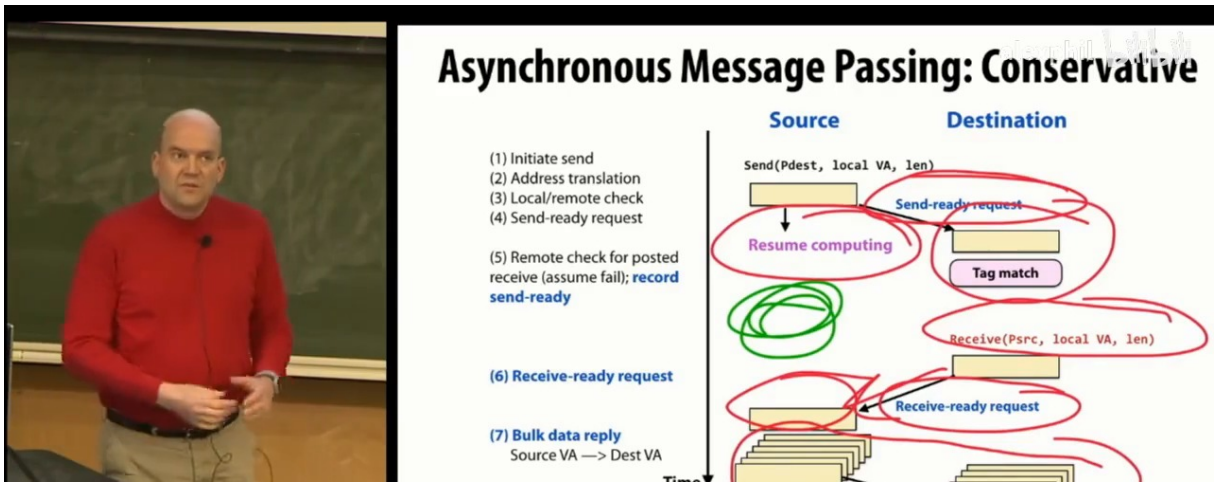


图 12: 异步消息传递（保守）：先确认再发送[12]

讲者说接收方侧就难得多。因为如果东西持续涌入，你很难回过头去说”嘿，等一下，你那边别再给我发了”——因为它已经在给你发了很多东西。

页面标记是一种”写时重定向”技巧

接收方把正在使用的页面标记为不可写，发送方的数据便只能落到其他位置。这样接收方可以安全地继续使用当前页面，而不必担心被覆盖。代价是多了一层页级映射与异常处理。

他提到一个常用的技巧：把你正在用的那个页面标记出来，让发送方写到别处去。但这也带来一个重要的正确性问题——如果你在”执行了 send”与”数据真正传输”之间写了那个地址，就会出问题。

[11] 视频画面时间区间：00:21:40–00:21:55。若为裁剪图，时间区间仍对应原视频画面。
[12] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

乐观异步的风险在于接收方无法回溯

发送方缓冲耗尽时它可以自我节流，这是本地决策；但接收方一旦被涌入的数据填满，就无法让已经在途的消息停下。因此保守方案必须事先获得许可，这正是信用机制要解决的问题。

接收方要应对涌入，除了信用机制还有另一类做法：把数据直接写入最终目的地。若接收方事先知道消息将放到哪个缓冲区，就不必先暂存再复制，而可以让发送方一次写到那里。这正是 eager 协议在 rendezvous 之后能做到零额外缓冲的原因。但要让接收方事先知道目的地，就必须有一次往返确认——于是又回到延迟与缓冲的取舍上。因此这两类方案不是互相替代，而是分别优化延迟与空间：乐观方案省掉往返但需要缓冲，保守方案省掉缓冲但需要往返。信用机制的价值就在于它允许大部分发送走乐观路径，只在信用耗尽时才退化为等待。

6.3 本章小结

乐观异步下，发送方缓冲耗尽会阻塞，形成自然流控，而且这在本地容易处理。接收方则难得多：数据已在涌入，无法回溯要求对方停止。常用的页面标记技巧还引出”send 与传输之间写入” 的正确性问题。

7 信用式混合方案

7.1 预分配与通告

讲者提出混合思路：能否在安全的时候享受乐观异步的速度，而不陷入可怕的缓冲问题?

做法是信用式方案：在接收方为所有发送方预分配一定量的空间。然后消息传递层会向它们通告” 已经为你预留了多少空间”。

他举例说比如四个字节或若干千字节。这个额度不需要很大就能显著减少往返等待的次数：只要发送方在一次突发中不超出额度，它就可以连续发送而不必等待确认，而接收方只需为每个发送方预留这一小块空间，总开销随参与者数量线性增长但系数很小。

7.2 跟踪用量

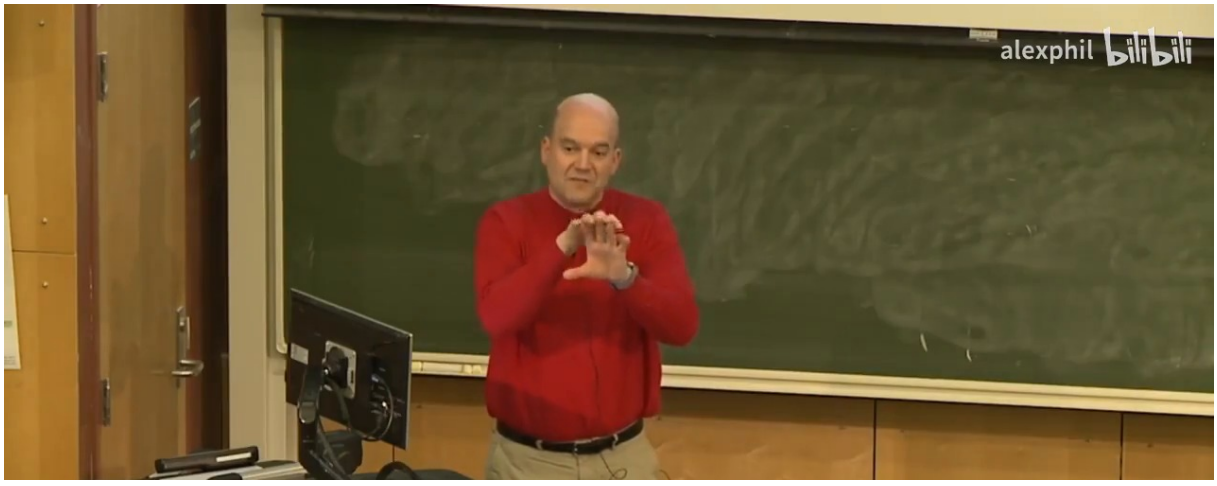


图 13: 信用式方案：通告可用空间并跟踪已用量[13]

然后发送方在尝试发消息时，会跟踪自己已经用掉了多少那部分空间。

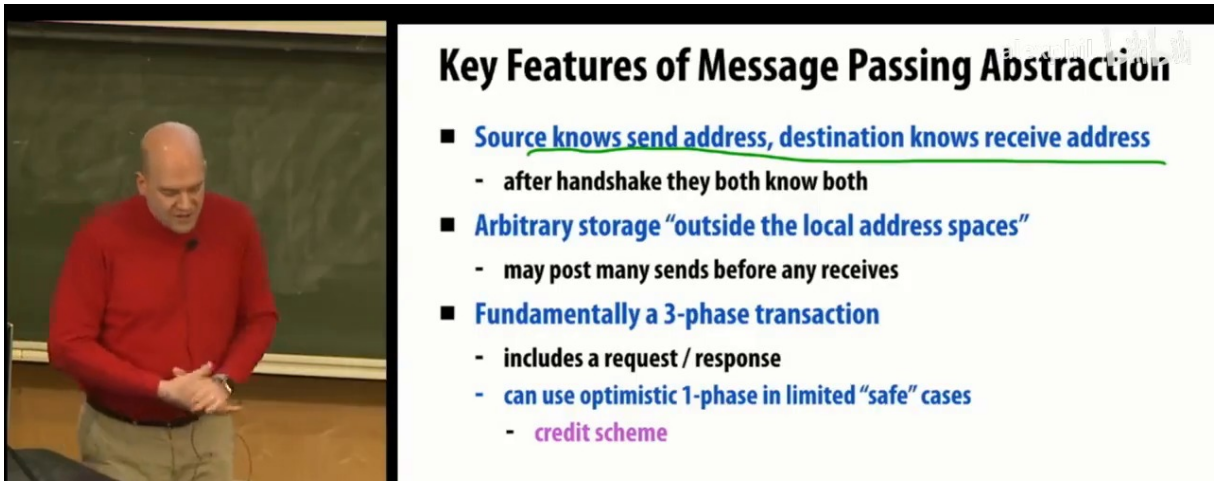


图 14: 消息传递抽象的关键特征：发送方知道接收方身份[14]

%

只要还有信用，它就可以继续以乐观方式发送，不必等待往返确认；一旦信用耗尽，它就必须等待接收方归还信用。

信用额度与等待频率的关系可近似为：

$$N_{\text{wait}} \approx \frac{B_{\text{total}}}{C}$$

- B_{total}：一次通信阶段要发送的总字节数；
- C：预分配的信用额度（字节）；
- N_{wait}：该阶段中发送方需要等待信用归还的次数；
- 额度越大，等待次数越少，但接收方需预留的缓冲也越大。

$$\text{可用信用} = \text{预分配额度} - \text{已发送未确认字节}$$

- 预分配额度：接收方为该发送方预留的缓冲上限；
- 已发送未确认字节：已经发出但接收方尚未消费的数据量；
- 只要可用信用为正，发送方无需等待确认即可继续发送。

信用机制把”全局流控”变成”逐对局部记账”

接收方不必在涌入时喊停，而是在建立连接时就声明”你能发多少”。发送方据此自行记账，用尽则自停。这把一个需要全局协调的问题化成了每个发送方本地可判定的条件，因此既安全又不必每次往返。

[13] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[14] 视频画面时间区间：00:29:30–00:29:45。若为裁剪图，时间区间仍对应原视频画面。

信用额度的大小需要权衡。额度设得大，发送方可以在长时间内不等待，但接收方必须为每个可能的发送方预留相应空间，总缓冲需求随参与者数量线性增长。额度设得小，接收方的缓冲压力小，但发送方会更频繁地等待信用归还，通信效率下降。因此实际系统常按连接建立时的可用内存与预期通信模式动态协商额度，并在运行中根据实际吞吐调整。这个调参过程与第 19 讲讨论的缓冲深度问题是同一个数学结构：都需要在「空间」与「等待」之间找到平衡点。

此外，信用归还的粒度也值得注意：若每消费一小段就归还一点，通信开销会累积；若只在整块消费完后归还，发送方又可能等待过久。实际实现常取折中，按固定批次归还。

7.3 本章小结

信用式方案在接收方为每个发送方预分配空间并通告额度，发送方跟踪已用量，只要信用为正就可持续乐观发送，用尽则等待归还。它把全局流控变成逐对局部记账，因此兼顾了速度与安全。

8 总结与延伸

讲者在结尾结束了这一部分，随后由另一位教授继续。

把全讲串起来，这是一条从” 消息传递为何容易实现” 到” 如何在速度与安全之间取舍” 的路径。消息传递的原语是把数据从发送方私有地址空间复制到接收方，它只需足够快的互连，不需要缓存一致性，因此在云计算刀片间也广泛使用——这是” 硬件成本低、软件成本高” 的不对称。共享地址空间靠读把数据拉进消费方缓存，由硬件做地址转换，本质是请求-响应协议，不需要额外缓冲，前提是双方都能通过地址访问同一份数据。消息传递有两种语义：同步让发送方停顿到接收方收到，异步则缓冲后继续。直觉偏向异步，但异步的不停顿以缓冲空间为前提。消息可达千字节至兆字节，小消息开销大因此应合并。eager 协议分三段：通知（含类型）、回应地址、DMA 传输，它不需要额外缓冲但至少三跳。乐观异步下发送方缓冲耗尽会阻塞并形成自然流控，这在本地易处理；接收方则难得多，因为数据已在涌入而无法回溯。最后，信用式混合方案在接收方为每个发送方预分配空间并通告额度，发送方跟踪已用量，用尽则等待归还——把全局流控变成逐对局部记账。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- ” 异步不停顿” 是有条件的。它的前提是缓冲空间可用；一旦耗尽，发送方仍会阻塞。因此异步的真实收益是” 把阻塞推迟并集中”，而不是消除阻塞。
- eager 协议的零额外缓冲与三跳延迟是一对交换。它把缓冲责任推给地址空间本身，代价是每次发送都要一次往返。因此它对大消息友好、对小消息不友好。
- 发送方与接收方的流控难度不对称。发送方可以本地自停，接收方却无法让在途数据回头。这个不对称正是信用机制存在的理由。
- 信用机制把全局问题化为局部记账。接收方在建立连接时声明额度，发送方自行记账，因此既避免全局协调，又不必每次往返确认。这是分布式流控的通用模式。

这堂课的边界在于它聚焦数据搬运与缓冲策略，未涉及路由算法、网络拓扑对消息延迟的影响，也未讨论 MPI 的具体实现细节。

两个可以直接从本讲数据提出的下一步问题：其一，信用额度的大小应当如何选择？过大则接收方缓冲压力大，过小则发送方频繁等待归还。其二，eager 协议的三跳延迟对延迟敏感的小消息不利，那么是否存在只针对小消息优化的第三条路径？